

RailKeeper Code-Audit und offene Baustellen

Stand: 04.06.2026

Arbeitsverzeichnis: C:\Users\droth\Documents\GitHub\RailKeeper

Lokale App: <http://localhost:8082>

Geprüfter Umfang

- Backend: API-Router, ECoS-Integration, Artikelsuche, Fahrzeug-Uploads, Backup/Stammdaten-Import, OpenAPI-Vertrag.
- Frontend: Fahrzeuge, Uploads, Ersatzteile, Import/Export, Einstellungen, i18n, AppSelect und relevante CSS-Flächen.
- Struktur: Top-Level-Ordner, untracked Dateien, ignorierte Runtime-/Build-Artefakte, Dokumentation und PDF-Tooling.
- Automatisierte Scans: Mojibake/kaputte Umlaute, TODO/FIXME/Debug-Marker, unfertige UI-Texte, auffällig laute Typografie.

Bereinigt

- Deutsche i18n-Texte repariert: kaputte UTF-8-Artefakte wurden in echte Umlaute und Zeichen zurückgeführt.
- Backend-Problemtexte geglättet: gross, ungueltig, unvollstaendig, geloescht und Mojibake wurden in lesbare deutsche Meldungen geändert.
- UI-Status ECoS-Import noch nicht final in ECoS-Import in Prüfung umformuliert.
- Ersatzteilsuche fachlich geschärft: lokale passende Ersatzteillisten werden zuerst gelesen, Webshop-Müll wird enger herausgefiltert.
- PDF-Auswertung verbessert: größere PDF-Content-Streams, echte Textobjekte und geteilte Tabellenzeilen werden berücksichtigt.
- Typografie in Uploads, Ersatzteilen, Import/Export und Digitalzentralen beruhigt: weniger 850/900-Gewichte in Hilfstexten, Tabellen und Labels.
- Historisches Übergabedokument repariert: Markdown-Quelle enthält wieder echte Umlaute.
- PDF-Renderer generisch benannt: `tools/render_markdown_pdf.py` ersetzt das session-spezifische `render_handover_pdf.py`.
- OpenAPI-Vertrag um die neue Route für Ersatzteilver schläge ergänzt.

Strukturprüfung

- `data/`, `.cache/`, `frontend/node_modules/`, `frontend/dist/` und `docs/*.pdf` sind in `.gitignore` abgedeckt. Lokale große Dateien bleiben damit Runtime-/Build-Artefakte.
- `backend/internal/ecos/` ist ein sinnvoller eigener Paketbereich für ECoS-Parsing und TCP-Client.
- `frontend/src/features/vehicles/` ist funktional stark gewachsen. Die Aufteilung in Tabs ist gut, `VehiclesView.tsx` bleibt aber die zentrale Koordinationsdatei und ist weiter zu groß.
- `frontend/src/styles/` ist nachvollziehbar nach Flächen getrennt, enthält aber noch viele sehr spezifische Selektoren und teils hohe Font-Weights.
- Alte Kompatibilitätspfade mit `legacy` in LocalStorage-Migrationen sind bewusst und sollten bleiben, bis genügend Installationen migriert sind.

Offene Baustellen

- 1 `VehiclesView.tsx` weiter zerlegen

Die Datei koordiniert Suche, Uploads, Ersatzteile, CVs, Fahrkurve und Dialoge. Nächster sauberer Schritt: Hooks oder Controller pro Tab.

- 1 Ersatzteil-PDFs ohne Textlayer

Die aktuelle Auswertung liest textbasierte PDFs. Gescannte Ersatzteilblätter brauchen OCR oder einen externen PDF-Textdienst.

- 1 PDF-Tabellen allgemeiner machen

PIKO-ähnliche Tabellen werden jetzt erkannt. Roco, Märklin, Tillig und ESU sollten mit echten Beispieldateien getestet und als Fixtures abgesichert werden.

- 1 Websuche für Ersatzteile weiter begrenzen

Webshop-HTML ist enger gefiltert, aber bleibt grundsätzlich unsicher. Besser wäre eine Quellen-Priorisierung nach Herstellerdokumenten vor Shopseiten.

- 1 Digitalzentralen-Sync

ECoS ist lesend nutzbar. Schreibender Sync, Konfliktauflösung und echte Gerätesteuerung bleiben bewusst Folgearbeiten.

- 1 Z21/CS3-Adapter

UI und Datenmodell sind provider-offen vorbereitet, Backend-Parser und Importpfade für Z21 und CS3 fehlen noch.

- 1 Benutzerbereich und 2FA

Passwortverwaltung, SMTP-Reset und Rollen sind vorhanden. 2FA ist sichtbar vorbereitet, aber Backend-Erzwingung fehlt.

- 1 Typografie-System formalisieren

Viele Stellen sind geglättet. Für dauerhaft ruhige UI sollte ein kleines Gewicht-/Größenraster als CSS-Konvention dokumentiert werden.

- 1 Line-Endings vereinheitlichen

Git meldet bei mehreren Frontend-Dateien CRLF/LF-Normalisierung. `.gitattributes` sollte für TS/TSX/CSS/MD eindeutige LF-Regeln setzen.

- 1 Dokumentations-PDFs

`docs/*.pdf` ist ignoriert. Wenn Audit-PDFs versioniert werden sollen, müssen sie bewusst mit `git add -f` aufgenommen werden.

Prüfergebnis

- Keine user-facing Mojibake-Treffer im App-Code gefunden.
- Keine aktiven `TODO`, `FIXME`, `DEBUG`, `debugger` oder `console.log`-Treffer im App-Code gefunden.
- Verbleibende Mojibake-Bytefolge in `article_search.go` ist absichtlich Teil des Reparatur-Erkenners.
- Große lokale Artefakte liegen in ignorierten Ordnern und werden nicht automatisch versioniert.